ELITEFORM

Documentación Técnica Completa

Guía para desarrolladores — Flutter + Firebase + Dart

Versión 1.0 • 2025

1. Introducción al Proyecto

EliteForm es una aplicación móvil de fitness desarrollada con Flutter y conectada a Firebase. Este documento explica de manera completa el código existente para que cualquier miembro del equipo pueda modificar, extender o depurar el proyecto sin necesidad de haberlo escrito desde cero.

1.1 Tecnologías utilizadas

Tecnología	¿Para qué se usa?
`Flutter`	Framework de Google para crear apps móviles con un solo código fuente (Android e iOS)
`Dart`	Lenguaje de programación que usa Flutter. Similar a JavaScript pero con tipos estáticos
`Firebase Auth`	Servicio de autenticación: maneja el login y registro de usuarios
`Cloud Firestore`	Base de datos NoSQL en la nube. Guarda perfiles de usuarios y datos de la app
`FlutterFire CLI`	Herramienta que conectó Firebase con el proyecto y generó firebase_options.dart

1.2 ¿Cómo instalar y ejecutar el proyecto?

1. Instalar Flutter SDK desde https://flutter.dev/docs/get-started/install
2. Abrir el proyecto en VS Code o Android Studio
3. Ejecutar `flutter pub get` en la terminal para descargar dependencias
4. Conectar un emulador o dispositivo físico y correr `flutter run`

IMPORTANTE: El archivo firebase_options.dart contiene las credenciales de Firebase. Nunca lo compartas públicamente ni lo subas a GitHub sin protegerlo con .gitignore o variables de entorno.

2. Estructura del Proyecto

El proyecto está organizado en carpetas según su responsabilidad. Esta separación se llama arquitectura por capas y hace el código más fácil de mantener.

Carpeta / Archivo	Responsabilidad
`lib/models/`	Define la forma de los datos. Ejemplo: cómo es un objeto 'Usuario'
`lib/pages/`	Contiene todas las pantallas visibles de la app (el 'frontend')
`lib/services/`	Lógica para comunicarse con Firebase (auth, Firestore)
`lib/widgets/`	Componentes visuales reutilizables (fondos, listas, etc.)
`lib/main.dart`	Punto de entrada de la app. Inicializa Firebase y define el tema visual
`lib/firebase_options.dart`	Credenciales generadas automáticamente por FlutterFire CLI

2.1 Diagrama de flujo de la app

Cuando el usuario abre la app, el flujo es el siguiente:

```
main.dart
   │
   ▼
LoginPage  ──── admin@admin.com ────►  AdminPage (lista de usuarios)
   │
   │  cualquier otro usuario
   ▼
HomePage (barra de navegación inferior)
   │
   ├── RutinaPage
   ├── DietaPage
   ├── EjerciciosPage
   ├── EventosPage
   └── PerfilPage
```

3. Conceptos Fundamentales de Dart y Flutter

3.1 StatelessWidget vs StatefulWidget

Toda pantalla en Flutter es un Widget. La diferencia clave es si la pantalla necesita actualizarse o no:

Concepto	Descripción
`StatelessWidget`	La pantalla es estática. Se dibuja una vez y no cambia. Ejemplo: RutinaPage, DietaPage
`StatefulWidget`	La pantalla puede cambiar. Tiene un 'estado' interno. Ejemplo: HomePage, PerfilPage, LoginPage
`State<T>`	La clase que acompaña a un StatefulWidget. Aquí se guardan los datos que pueden cambiar
`setState()`	Método que le dice a Flutter 'algo cambió, vuelve a dibujar la pantalla'
`initState()`	Se ejecuta UNA SOLA VEZ cuando la pantalla aparece. Ideal para cargar datos iniciales
`build()`	Se ejecuta cada vez que Flutter necesita dibujar el widget. Debe retornar un Widget

Ejemplo en el proyecto: PerfilPage

PerfilPage es StatefulWidget porque el usuario puede alternar entre modo ver y modo editar:

```
class PerfilPage extends StatefulWidget {
  // StatefulWidget: la pantalla puede cambiar
}

class _PerfilPageState extends State<PerfilPage> {
  bool editando = false;  // <-- Este es el 'estado'

  void cambiarModo() {
    setState(() {
      editando = true;  // Cambia el estado
    });
    // Flutter vuelve a llamar build() automáticamente
  }
}
```

3.2 async / await — Operaciones que toman tiempo

Firebase tarda tiempo en responder. En Dart usamos async/await para esperar sin bloquear la app:

```
// SIN async/await (MAL - bloquea la app)
void cargar() {
  var datos = Firebase.obtener();  // La app se congela esperando
}
```

```
// CON async/await (BIEN - la app sigue funcionando)
Future<void> cargar() async {
  var datos = await Firebase.obtener();  // Espera sin congelar
  setState(() { /* actualiza la pantalla */ });
}
```

La palabra `Future` significa 'esto me dará un resultado en el futuro'. Es como una promesa (`Promise` en JavaScript).

3.3 try / catch — Manejo de errores

Cuando llamas a Firebase, pueden ocurrir errores (sin internet, contraseña incorrecta, etc.). Se capturan con try/catch:

```
try {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
  // Si llega aquí, el login fue exitoso
} on FirebaseAuthException catch (e) {
  // Si Firebase lanza un error, entra aquí
  showDialog(/* muestra el error al usuario */);
}
```

3.4 Widgets principales utilizados en el proyecto

Widget	Uso en el proyecto
`Scaffold`	Estructura base de una pantalla: tiene appBar, body, bottomNavigationBar
`AppBar`	Barra superior con título e íconos. Configurada globalmente en main.dart
`Column / Row`	Organiza widgets en vertical (Column) u horizontal (Row)
`ListView.builder`	Lista eficiente que solo renderiza los elementos visibles
`TextField`	Campo de texto para entrada del usuario. Se controla con TextEditingController
`ElevatedButton`	Botón elevado. Ya tiene estilo naranja configurado en main.dart
`StreamBuilder`	Escucha cambios en tiempo real de Firestore y reconstruye la UI automáticamente
`Navigator`	Controla la navegación entre pantallas (push, pop, pushReplacement)
`BottomNavigationBar`	Barra de navegación inferior con íconos. Usada en HomePage
`Card`	Contenedor con sombra y bordes redondeados. Usado en UsersList

Widget	Uso en el proyecto
`CircleAvatar`	Imagen circular. Usada como ícono de usuario en la lista de admin
`ListTile`	Fila con ícono, título y subtítulo. Usado en PerfilPage para mostrar datos
`ScaffoldMessenger`	Muestra mensajes temporales (SnackBar) al fondo de la pantalla
`AlertDialog`	Ventana emergente modal. Usada para errores y confirmaciones

4. Archivos del Proyecto — Explicación Detallada

4.1 main.dart — El punto de inicio

Este es el primer archivo que se ejecuta. Hace dos cosas: inicializar Firebase y definir el estilo visual global de toda la app.

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // ^ SIEMPRE debe ir antes de Firebase.initializeApp()
  // Asegura que Flutter esté listo antes de hacer operaciones asíncronas

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
    // ^ Lee las credenciales correctas según el dispositivo (Android/iOS/Web)
  );

  runApp(const MyApp());
}
```

El ThemeData en main.dart define el estilo de TODA la app. Si cambias un color aquí, cambia en todas las pantallas:

Propiedad del tema	Efecto
`scaffoldBackgroundColor`	Color de fondo de todas las pantallas (azul muy oscuro: #0F172A)
`colorScheme.primary`	Color principal: naranja. Afecta botones, íconos activos, etc.
`appBarTheme`	Estilo de la barra superior en todas las páginas
`elevatedButtonTheme`	Estilo de todos los ElevatedButton: naranja, texto negro, bordes redondeados
`inputDecorationTheme`	Estilo de todos los TextField: fondo #1E293B, sin borde, íconos naranjas

4.2 login_page.dart — Pantalla de Login

Maneja el acceso a la app. Tiene dos rutas de autenticación:

- Administrador: `admin@admin.com` / `123456` — credenciales hardcoded (escritas directamente en el código), no pasan por Firebase Auth
- Usuarios normales: Se autentican con Firebase Auth

ATENCIÓN: La contraseña de admin está escrita en texto plano en el código. Esto es un riesgo de seguridad. Para producción, se recomienda verificar el rol del usuario en Firestore en lugar de comparar strings.

Componentes clave de LoginPage:

Elemento	Descripción
`emailController`	TextEditingController que captura lo que el usuario escribe en el campo email
`passwordController`	TextEditingController para la contraseña
`loginUser()`	Función async que maneja la lógica: primero verifica admin, luego llama Firebase
`showError()`	Muestra un AlertDialog con el mensaje de error recibido
`AppBackground`	Widget personalizado que aplica el degradado de fondo (importado de widgets/)

4.3 register_page.dart — Registro de nuevos usuarios

Crea cuentas nuevas en Firebase Auth Y guarda los datos adicionales en Firestore. Pasos que sigue:

5. Valida que el teléfono tenga exactamente 10 dígitos
6. Llama a Firebase Auth para crear el usuario (email + contraseña)
7. Guarda nombre, teléfono y correo en Firestore en `colección 'usuarios'` usando el UID como ID del documento
8. Navega a HomePage

Validación del teléfono — se hace en dos capas:

- `maxLength: 10` — limita la escritura a 10 caracteres
- `FilteringTextInputFormatter.digitsOnly` — bloquea letras y símbolos en el teclado
- Validación lógica: `if (telefonoController.text.length != 10) return;` — evita llamar a Firebase si el dato es inválido

4.4 home_page.dart — Pantalla principal con navegación

HomePage es el contenedor principal después del login. Maneja la barra de navegación inferior:

```
// Lista de páginas — el índice corresponde al ícono en la barra inferior
final List<Widget> _pages = const [
  RutinaPage(),     // índice 0
  DietaPage(),      // índice 1
  EjerciciosPage(), // índice 2
  EventosPage(),    // índice 3
  PerfilPage(),     // índice 4
];

// Cuando el usuario toca un ícono:
void _onItemTapped(int index) {
  setState(() {
    _selectedIndex = index;  // Cambia cuál página se muestra
  });
```

```
}

// En el body, solo muestra la página del índice actual:
body: _pages[_selectedIndex],
```

Para agregar una nueva sección: 1) Crea el archivo de la página. 2) Agrégala a la lista _pages. 3) Agrega un BottomNavigationBarItem en la lista items del BottomNavigationBar.

4.5 perfil_page.dart — Perfil del usuario

Página compleja que permite ver, editar y eliminar la cuenta. Tiene dos modos controlados por la variable bool editando:

Estado / Método	Comportamiento
`editando = false`	Modo vista: muestra ListTile con nombre, teléfono, correo. Botones de editar y eliminar
`editando = true`	Modo edición: muestra TextField editables para nombre y teléfono. Correo bloqueado
`cargarDatos()`	Se llama en initState(). Lee los datos del usuario desde Firestore
`actualizarPerfil()`	Valida el teléfono y actualiza el documento en Firestore con nombre y teléfono nuevos
`eliminarCuenta()`	Muestra confirmación, borra el documento de Firestore y elimina el usuario de Auth

4.6 admin_page.dart — Panel de administrador

Pantalla simple solo accesible con las credenciales de admin. Muestra el widget UsersList que lista todos los usuarios registrados en Firestore.

El método `_logout()` usa `AuthService` para cerrar sesión y redirige a LoginPage. Nota el uso de `context.mounted` — esto es una buena práctica para evitar errores si la pantalla ya no existe cuando Firebase responde.

4.7 Páginas 'Próximamente' — RutinaPage, DietaPage, EjerciciosPage, EventosPage

Estas cuatro páginas son actualmente StatelessWidget con solo un Text de placeholder. Son los lugares donde el equipo desarrollará las funcionalidades principales:

```
class RutinaPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Rutinas próximamente'),
      // TODO: Reemplazar con la implementación real
    );
  }
```

```
}
```

Ver Sección 6 para guía paso a paso de cómo implementar estas páginas con datos reales de Firebase.

5. Servicios y Modelos

5.1 user_model.dart — El modelo de datos

Define la estructura de un usuario en la app. Es como un 'molde' que convierte datos crudos de Firestore en un objeto Dart:

```
class UserModel {
  final String id;
  final String nombre;
  final String correo;
  final String telefono;

  // Constructor: forma de crear un UserModel
  UserModel({ required this.id, required this.nombre, ... });

  // Factory: convierte un Map (datos de Firestore) en UserModel
  factory UserModel.fromMap(Map<String, dynamic> data) {
    return UserModel(
      id: data['id_usuario'],   // Nombre de campo en Firestore
      nombre: data['nombre'],
      correo: data['correo'],
      telefono: data['telefono'],
    );
  }
}
```

Los campos en Firestore son: id_usuario, nombre, correo, telefono, fecha_registro.

5.2 auth_service.dart — Servicio de autenticación

Encapsula todas las operaciones de Firebase Auth en métodos reutilizables:

Método	Función
`register(email, password)`	Crea un nuevo usuario en Firebase Auth. Retorna UserCredential
`login(email, password)`	Inicia sesión. Retorna UserCredential si es exitoso
`logout()`	Cierra la sesión del usuario actual
`currentUser`	Propiedad que retorna el User? actual (null si no hay sesión)

Este servicio existe pero login_page.dart y register_page.dart llaman a FirebaseAuth.instance directamente en algunos lugares. Si quieres consistencia, considera refactorizar para usar siempre AuthService.

5.3 firestore_service.dart — Servicio de base de datos

Centraliza las operaciones CRUD (Create, Read, Update, Delete) de Firestore:

Método	Operación
`createUser(uid, nombre, correo, telefono)`	Crea un documento nuevo en la colección 'usuarios'
`updateUser(uid, nombre, telefono)`	Actualiza nombre y teléfono de un documento existente
`deleteUser(uid)`	Elimina el documento del usuario de Firestore
`getUsers()`	Retorna un Stream que emite actualizaciones en tiempo real de todos los usuarios

Al igual que AuthService, FirestoreService existe pero las páginas (perfil_page, register_page) llaman a Firestore directamente. Para el futuro se recomienda usar siempre los servicios para mayor mantenibilidad.

5.4 app_background.dart — Widget de fondo

Widget reutilizable que aplica el degradado de fondo azul oscuro. Solo se usa en LoginPage actualmente:

```
// Uso:
return Scaffold(
  body: AppBackground(  // Envuelve cualquier pantalla
    child: Center(
      child: Text('Mi contenido'),
    ),
  ),
);
```

5.5 users_list.dart — Lista de usuarios en tiempo real

Widget que usa StreamBuilder para mostrar usuarios de Firestore en tiempo real. Es el ejemplo más complejo del proyecto:

```
StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
    .collection('usuarios').snapshots(),
  // ^ Cada vez que Firestore cambia, Flutter reconstruye el widget

  builder: (context, snapshot) {
    // Estado 1: Error en la conexión
    if (snapshot.hasError) return Text('Error');

    // Estado 2: Cargando (conexión en progreso)
    if (snapshot.connectionState == ConnectionState.waiting)
      return CircularProgressIndicator();

    // Estado 3: Sin datos
    if (!snapshot.hasData || snapshot.data!.docs.isEmpty)
      return Text('No hay usuarios');

    // Estado 4: Datos disponibles
```

```
    final users = snapshot.data!.docs;
    return ListView.builder(
      itemCount: users.length,
      itemBuilder: (context, index) {
        var user = users[index];
        return Card(child: ListTile(
          title: Text(user['correo']),
          subtitle: Text(user['id_usuario']),
        ));
      },
    );
  },
)
```

6. Guía Práctica — Cómo Modificar el Proyecto

6.1 Implementar RutinaPage con datos reales

Objetivo: mostrar una lista de rutinas guardadas en Firestore en lugar del texto 'próximamente'.

Paso 1 — Crear la colección en Firestore. En la consola de Firebase, crea una colección llamada 'rutinas' con documentos que tengan estos campos:

Campo	Tipo y ejemplo
`nombre`	String — Ej: 'Rutina de pecho'
`descripcion`	String — Ej: 'Flexiones, press banca, aperturas'
`duracion_min`	Number — Ej: 45
`dificultad`	String — Ej: 'Intermedio'

Paso 2 — Convertir RutinaPage a StatefulWidget y agregar un StreamBuilder:

```
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

class RutinaPage extends StatelessWidget {
  const RutinaPage({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<QuerySnapshot>(
      stream: FirebaseFirestore.instance
        .collection('rutinas').snapshots(),
      builder: (context, snapshot) {
        if (snapshot.hasError)
          return const Center(child: Text('Error al cargar'));
        if (snapshot.connectionState == ConnectionState.waiting)
          return const Center(child: CircularProgressIndicator());
        if (!snapshot.hasData || snapshot.data!.docs.isEmpty)
          return const Center(child: Text('No hay rutinas aún'));

        final rutinas = snapshot.data!.docs;
        return ListView.builder(
          itemCount: rutinas.length,
          itemBuilder: (context, index) {
            var r = rutinas[index];
            return Card(
              margin: const EdgeInsets.all(10),
              child: ListTile(
                leading: const Icon(Icons.fitness_center,
                  color: Colors.orangeAccent),
                title: Text(r['nombre']),
                subtitle: Text('${r['duracion_min']} min ·
${r['dificultad']}'),
              ),
            );
          },
        );
      },
    );
  }
```

```
}
```

6.2 Agregar una nueva pantalla completa

Ejemplo: agregar una pantalla de 'Progreso' con gráficas o estadísticas.

9. Crear el archivo lib/pages/progreso_page.dart con la estructura básica:

```
import 'package:flutter/material.dart';

class ProgresoPage extends StatelessWidget {
  const ProgresoPage({super.key});

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Text('Mi progreso'),
    );
  }
}
```

10. Importar en home_page.dart y agregar a la lista:

```
import 'progreso_page.dart';

final List<Widget> _pages = const [
  RutinaPage(),
  DietaPage(),
  EjerciciosPage(),
  EventosPage(),
  ProgresoPage(),  // <-- Agrega aquí
  PerfilPage(),
];
```

11. Agregar el ícono en el BottomNavigationBar (en home_page.dart):

```
BottomNavigationBarItem(
  icon: Icon(Icons.bar_chart),
  label: 'Progreso',
),
```

6.3 Mostrar datos del usuario actual en cualquier pantalla

Para obtener el UID del usuario logueado y cargar sus datos de Firestore:

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

Future<void> cargarMisDatos() async {
  // 1. Obtener el usuario autenticado actual
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;  // No hay sesión activa

  // 2. Leer su documento de Firestore usando su UID
  final doc = await FirebaseFirestore.instance
    .collection('usuarios')
    .doc(user.uid)  // El UID de Auth coincide con el ID del doc
    .get();

  // 3. Acceder a los campos
```

```
  String nombre = doc['nombre'];
  String telefono = doc['telefono'];
}
```

6.4 Navegar entre pantallas

Flutter usa un sistema de 'stack' de pantallas. Estos son los tres métodos más comunes:

Método	Comportamiento
`Navigator.push()`	Agrega la nueva pantalla ENCIMA de la actual. El usuario puede regresar con el botón atrás
`Navigator.pushReplacement()`	Reemplaza la pantalla actual. No hay forma de regresar. Usado en login → home
`Navigator.pop()`	Elimina la pantalla actual y regresa a la anterior

```
// push — ir a detalles y poder regresar
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const DetalleRutinaPage()),
);

// pushReplacement — no dejar ruta de regreso
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const LoginPage()),
);

// pop — cerrar la pantalla actual
Navigator.pop(context);

// pop con resultado — útil para diálogos
Navigator.pop(context, true);  // Devuelve 'true' a quien abrió esta pantalla
```

6.5 Escribir datos en Firestore

Para guardar datos nuevos (por ejemplo, una rutina creada por el usuario):

```
Future<void> guardarRutina(String nombre, int duracion) async {
  final user = FirebaseAuth.instance.currentUser;

  await FirebaseFirestore.instance
    .collection('rutinas')
    .add({                          // .add() genera ID automático
      'nombre': nombre,
      'duracion_min': duracion,
      'uid_usuario': user!.uid,      // Vincular al usuario actual
      'fecha_creacion': FieldValue.serverTimestamp(),
    });

  // Para usar un ID personalizado:
  // .doc('mi-id-personalizado').set({...})
}
```

7. Firebase — Referencia Rápida

7.1 Estructura de datos en Firestore

Actualmente el proyecto solo usa la colección 'usuarios'. Estructura sugerida para crecer:

```
Firestore
│
├── usuarios/              (colección existente)
│   └── {uid}/             (documento por usuario)
│       ├── id_usuario: string
│       ├── nombre: string
│       ├── correo: string
│       ├── telefono: string
│       └── fecha_registro: timestamp
│
├── rutinas/               (colección sugerida)
│   └── {id}/
│       ├── nombre: string
│       ├── descripcion: string
│       ├── duracion_min: number
│       └── uid_usuario: string    (para filtrar por usuario)
│
├── eventos/               (colección sugerida)
│   └── {id}/
│       ├── titulo: string
│       ├── fecha: timestamp
│       └── descripcion: string
│
└── dietas/                (colección sugerida)
    └── {id}/
        ├── nombre: string
        └── uid_usuario: string
```

7.2 Reglas de seguridad de Firestore

En la consola de Firebase > Firestore > Reglas, se controla quién puede leer o escribir. Las reglas actuales del proyecto (por defecto durante desarrollo) probablemente permiten todo. Para producción, considera esto:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Solo usuarios autenticados pueden leer/escribir sus propios datos
    match /usuarios/{userId} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }
    // Solo usuarios autenticados pueden leer eventos
    match /eventos/{eventId} {
      allow read: if request.auth != null;
      allow write: if false;  // Solo el admin puede escribir
    }
  }
}
```

7.3 Errores comunes de Firebase Auth

Código de error	Causa
`user-not-found`	El correo no está registrado en Firebase Auth
`wrong-password`	La contraseña es incorrecta
`email-already-in-use`	Ya existe una cuenta con ese correo
`weak-password`	La contraseña tiene menos de 6 caracteres
`network-request-failed`	Sin conexión a internet
`requires-recent-login`	Para eliminar cuenta o cambiar email, el usuario debe haber iniciado sesión recientemente

8. Buenas Prácticas y Errores a Evitar

8.1 Problemas frecuentes

Práctica	Explicación
`context.mounted`	Antes de usar context después de un await, verifica if (context.mounted). Evita errores si la pantalla se cerró mientras esperabas
`const donde sea posible`	Flutter renderiza más rápido los widgets marcados con const. Agrégalo cuando el widget no cambia
`Dispose de controllers`	Los TextEditingController deben liberarse en dispose() para evitar memory leaks
`Evitar setState innecesario`	No llames setState() si solo cambias variables que no afectan el build()
`! (null assertion)`	Evita user!.uid a menos que estés seguro de que no es null. Prefiere user?.uid ?? ''

8.2 Cómo agregar correctamente un TextEditingController

```
class MiPagina extends StatefulWidget { ... }

class _MiPaginaState extends State<MiPagina> {
  // 1. Declarar el controller
  final miController = TextEditingController();

  // 2. Liberar cuando la pantalla se destruye
  @override
  void dispose() {
    miController.dispose();  // MUY IMPORTANTE
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: miController,
      // 3. Leer el valor cuando lo necesites:
      // miController.text
    );
  }
}
```

8.3 Checklist antes de hacer cambios

- ¿La pantalla necesita actualizarse? → Usar StatefulWidget. ¿No? → StatelessWidget
- ¿Llamas a Firebase? → Agregar async/await y try/catch
- ¿Usas context después de un await? → Verificar if (context.mounted)
- ¿Agregaste un TextEditingController? → Agregar dispose()

- ¿Navegas a una pantalla y no debes poder regresar? → pushReplacement
- ¿Los cambios visuales no aparecen? → Verificar que llamaste setState()

8.4 Recursos para aprender

Recurso	Descripción
`flutter.dev/docs`	Documentación oficial de Flutter en español disponible
`firebase.google.com/docs/flutter`	Guías oficiales de Firebase para Flutter
`pub.dev`	Repositorio de paquetes de Flutter/Dart
`dartpad.dev`	Editor online para practicar Dart sin instalar nada
`Flutter YouTube`	Canal oficial con tutoriales cortos de cada widget

Documentación generada para el equipo de desarrollo de **EliteForm**. Versión 1.0 — 2025.